

Image Fragment Reconstruction via Self-Supervised Learning

Task Summary and Algorithm Design

May 2, 2026

Contents

1	Task Recapitulation	2
1.1	Core Objective	2
1.2	Points That Required Clarification	2
2	Related Problems	3
2.1	Jigsaw Puzzle Solving	3
2.2	Instance Discrimination and Contrastive Learning	3
2.3	Deep Clustering	3
2.4	Summary of Similarities and Differences	4
3	Algorithm Outline and Decision Points	4
3.1	Overview	4
3.2	Stage 1: Data Preparation	4
3.3	Stage 2: Encoder Architecture	4
3.4	Stage 3: Contrastive Loss	5
3.5	Stage 4: Clustering	5
3.6	Stage 5: Evaluation Metrics	5
3.7	Summary of Decision Points	6
4	Non-Trivial Challenges and Limitations of the Naive Approach	6
4.1	The Core Difficulty: Visual Similarity Does Not Equal Source Identity . . .	6
4.2	Implications for Model Design	6
4.3	Summary	7

1 Task Recapitulation

1.1 Core Objective

The task is to develop a self-supervised machine learning model that groups a large collection of mixed image fragments back to their original source images. Concretely, the pipeline is as follows:

1. A data generator yields a sample of **10 images** ($64 \times 64 \times 3$ pixels each), with augmentation optionally applied before delivery.
2. Each image is cut into a **4×4 grid** of non-overlapping tiles, producing 16 fragments of $16 \times 16 \times 3$ pixels per image.
3. All 160 fragments ($10 \text{ images} \times 16 \text{ tiles}$) are **shuffled into a single unordered collection** — no label, no position information is retained.
4. The model must learn to **group these 160 fragments back into 10 clusters**, each corresponding to one source image.

1.2 Points That Required Clarification

Several aspects of the task were not immediately obvious and deserve explicit statement:

Self-supervised does not mean label-free throughout. The ground-truth grouping (which fragment came from which image) is known to the experimenter at all times — it is used to construct training pairs and to evaluate the model. What the model never receives is an explicit label. The supervision signal is derived from the data generation process itself, not from human annotation.

The goal is grouping, not spatial reconstruction. The model is not required to determine where each fragment was positioned within the original image (top-left, bottom-right, etc.). It only needs to identify which fragments originated from the same image. Think of sorting puzzle pieces back into their correct boxes without assembling the puzzles.

The generator yields augmented images, not augmented fragments. Augmentation is applied to the full 64×64 image *before* it is cut into fragments. Consequently, all 16 fragments derived from one generator call share a consistent (augmented) visual appearance. The augmentations applied are: rotation (up to 20°), horizontal and vertical shift (up to 10%), and channel shift (up to 0.2). These are defined in `data.py` via `init_augmentor()`.

Augmentation as an implicit constraint. Because augmentation randomly perturbs colours and positions, any model that relies on exact pixel values or absolute spatial coordinates will fail. The model is therefore forced to learn higher-level features — texture, pattern, and visual style — that remain stable across an image regardless of minor transformations.

The number of clusters is known. Since the experimenter controls how many images are fed into each sample (always 10), the number of clusters k is known in advance. Algorithms that require k as input (e.g. k -means) can therefore be used directly. In a real-world deployment where k is unknown, discovery-based methods (e.g. DBSCAN) would be required.

Balanced clusters. Because each image is always cut into exactly 16 fragments, the true clusters are perfectly balanced. This can be exploited by constraining the clustering step to produce balanced clusters of 16, which reduces the search space and can improve accuracy.

Infinite effective training data. Since augmentations are sampled randomly on every generator call, no two batches are identical. The training set is effectively infinite, and the experimenter controls training duration by choosing how many generator calls to make.

2 Related Problems

The task borrows from three well-established areas of machine learning research.

2.1 Jigsaw Puzzle Solving

Noroozi and Favaro (2016) proposed using jigsaw puzzle reassembly as a self-supervised pretext task: image patches are shuffled and a network is trained to predict their correct spatial arrangement. This is closely related to the present task in that both work with fragments of images without labels. The key difference is that jigsaw methods require recovering the *spatial position* of each fragment, whereas here only *source identity* (which image did this fragment come from) must be determined.

2.2 Instance Discrimination and Contrastive Learning

Chen et al. (2020) introduced SimCLR, a framework in which two augmented views of the same image are treated as a positive pair and all other images in the batch as negatives. The encoder is trained so that positive pairs have similar embeddings and negative pairs have dissimilar embeddings. This is essentially the training objective used in the present task, with fragments from the same source image playing the role of positive pairs.

He et al. (2020) proposed MoCo, which extends contrastive learning with a momentum encoder and a memory bank to increase the number of negatives available per update. MoCo is most beneficial when batch sizes are small. In the present task the number of negatives per batch is already manageable (at most 144 fragments from other images within a single 160-fragment sample), making the additional complexity of MoCo unnecessary.

2.3 Deep Clustering

Standard unsupervised image clustering methods learn embeddings and cluster assignments jointly. The present task is distinct in that the unit to be clustered is a *fragment*, not a whole image, and the cluster identity is ephemeral — it changes with every generator call and carries no semantic meaning beyond the current sample.

2.4 Summary of Similarities and Differences

	Jigsaw	SimCLR	This task
Works with fragments	✓		✓
Recovers spatial position	✓		
Contrastive objective		✓	✓
Consistent class identity across batches		✓	
Clustering at inference			✓

3 Algorithm Outline and Decision Points

3.1 Overview

The proposed pipeline has three stages:

1. **Data preparation** — fragment images and construct training pairs.
2. **Training** — learn an encoder that maps fragments to embeddings using a contrastive loss.
3. **Evaluation** — cluster the embeddings with k -means and measure grouping accuracy.

3.2 Stage 1: Data Preparation

Call the generator with `batch_size=10` and `augmentation=True`. Cut each of the 10 returned images into a 4×4 grid, yielding 160 fragments of $16 \times 16 \times 3$. Retain the source-image index for each fragment — this is used to construct training pairs and to evaluate, but is never passed to the model.

3.3 Stage 2: Encoder Architecture

Decision point: architecture. A small **CNN encoder** is the natural choice for 16×16 image patches. CNNs extract local spatial features (edges, textures, patterns) efficiently at this scale. Vision Transformers are designed for larger inputs and would be overkill here. An autoencoder could also produce embeddings via its bottleneck, but its reconstruction objective does not directly optimise for inter-group separation.

Decision point: embedding dimension. A dimension of 128–256 is standard for contrastive methods and provides enough capacity without making clustering unnecessarily difficult.

Decision point: projection head. Following Chen et al. (2020), a two-layer MLP projection head is added on top of the encoder during training and discarded at evaluation time. This is empirically shown to improve the quality of the encoder representations.

3.4 Stage 3: Contrastive Loss

The NT-Xent loss from SimCLR (Chen et al. 2020) is used. For each fragment i , its positive set consists of the other 15 fragments from the same source image. All remaining fragments in the batch serve as negatives. The loss encourages:

small intra-group distance and large inter-group distance

Decision point: distance metric. **Cosine similarity** is used as the distance measure, consistent with SimCLR. Cosine similarity measures the angle between embedding vectors, making it invariant to the magnitude of the embeddings.

Decision point: training duration. Since each generator call produces a unique augmented sample, training duration is controlled by the number of generator calls (iterations). This is a hyperparameter to tune; more iterations generally improve representations up to a point of diminishing returns.

3.5 Stage 4: Clustering

Decision point: clustering algorithm. **k -means** with $k = 10$ is used, since the number of source images per sample is known. The key constraint is that the clustering algorithm must use the **same distance metric as the contrastive loss**. Because cosine similarity was used during training, embeddings should be ℓ_2 -normalised before clustering — on the unit hypersphere, Euclidean distance and cosine distance are equivalent, so standard k -means applies correctly.

If Euclidean distance were used during training instead, ℓ_2 -normalisation would not be appropriate and standard k -means could be applied directly to unnormalised embeddings. Mismatching the training metric and the clustering metric would result in clusters that do not align with what the model actually learned.

Decision point: balanced clusters. Since the true cluster size is always exactly 16, the clustering can be constrained to produce balanced clusters. This reduces ambiguity and can improve accuracy.

3.6 Stage 5: Evaluation Metrics

Decision point: metric choice. Simple **cluster purity** measures the fraction of the dominant class in each cluster. It is intuitive but ignores how errors are distributed. For example, a cluster containing 60% fragments from image A and 40% from image B scores identically to a cluster containing 60% from image A, 20% from image B, and 20% from image C — despite the latter being more disordered.

Adjusted Rand Index (ARI) and **Normalised Mutual Information (NMI)** capture the full distribution of fragments across all clusters simultaneously and are therefore the preferred metrics. Both are standard in clustering evaluation and correct for chance agreement.

3.7 Summary of Decision Points

Decision	Choice	Alternatives
Encoder architecture	Small CNN	Autoencoder, ViT
Embedding dimension	128–256	Smaller / larger
Training objective	NT-Xent (SimCLR)	Triplet loss, BYOL
Distance metric	Cosine similarity	Euclidean, dot product
Clustering algorithm	k -means ($k = 10$)	DBSCAN, spectral
Cluster size constraint	Balanced (16 per cluster)	Unconstrained
Evaluation metric	ARI, NMI	Purity, accuracy
Training duration	Number of generator calls	Epochs over fixed dataset

4 Non-Trivial Challenges and Limitations of the Naive Approach

4.1 The Core Difficulty: Visual Similarity Does Not Equal Source Identity

The approach outlined in Section 3 is a reasonable starting point, but it rests on an assumption that deserves scrutiny. Standard contrastive learning methods such as SimCLR (Chen et al. 2020) work by treating two *augmented views of the same image* as a positive pair. Because both views are derived from the same image, they look very similar — and the contrastive loss succeeds by exploiting that visual similarity.

In the present task this assumption breaks down in two directions simultaneously:

- **Intra-image fragments can be visually dissimilar.** A 16×16 tile from the sky region of a landscape photograph looks nothing like a tile from the ground region of the same photograph. Yet the model is asked to place both close together in embedding space.
- **Inter-image fragments can be visually similar.** A sky tile from image A and a sky tile from image B may be nearly identical in colour and texture, yet the model is asked to push them apart.

This is a fundamental tension that a naive pairwise contrastive loss will struggle to resolve. The model cannot rely on low-level visual similarity as a proxy for source identity, because that proxy is unreliable in both directions.

4.2 Implications for Model Design

This tension suggests that the problem may require approaches beyond standard SimCLR:

Attention mechanisms. A transformer-based encoder could allow fragments to attend to each other within a sample, learning relational features rather than purely local ones. This would give the model a way to detect global consistency across fragments beyond what any single tile contains.

Clustering-oriented losses. Methods such as SwAV (Caron et al. 2020) or SCAN assign soft cluster assignments during training and use those assignments as part of the loss, making the training objective more directly aligned with the final clustering goal. These may be better suited to cases where pairwise visual similarity is not a reliable training signal.

Context as a feature. If spatial position is not required as output, it can still be used as an auxiliary signal during training. Predicting the relative position of two fragments from the same image is a pretext task (Noroozi and Favaro 2016) that implicitly requires the model to understand source identity.

4.3 Summary

The naive SimCLR approach is a well-justified starting point and may perform adequately when source images are visually diverse. However, the fundamental mismatch between visual similarity and source identity means that performance on hard cases — images with uniform or repetitive content, or datasets with many visually similar images — is likely to be poor. Recognising this limitation and discussing it explicitly, whether or not it is fully resolved in the implementation, is an important part of the intellectual contribution of the work.

References

- Caron, Mathilde et al. (2020). “Unsupervised Learning of Visual Features by Contrasting Cluster Assignments”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33, pp. 9912–9924.
- Chen, Ting et al. (2020). “A Simple Framework for Contrastive Learning of Visual Representations”. In: *International Conference on Machine Learning (ICML)*, pp. 1597–1607.
- He, Kaiming et al. (2020). “Momentum Contrast for Unsupervised Visual Representation Learning”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9729–9738.
- Noroozi, Mehdi and Paolo Favaro (2016). “Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles”. In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 69–84.